

UNIVERSIDAD CATOLICA DE ORIENTE

Facultad de Ingenieria - Compiladores

Claudio - Gramatica ampliada con reglas semanticas

Quiz 4 / Entrega 4

Lenguaje fuente: Claudio (palabras clave en espanol) | Lenguaje destino: Swift

Entregable PDF: producciones sintacticas originales en rojo y reglas semanticas en fondo ambar, segun el enunciado.

Presentado por: Juan David Cardenas Jimenez, Maria Jose Restrepo Ramirez, Jose Manuel Zuluaga F.

Semestre 1-2026

1. Alcance del entregable

Este documento presenta la version ampliada de la gramatica usada por Claudio para el Quiz 4. La ampliacion conserva las producciones sintacticas del lenguaje, pero incorpora reglas semanticas que verifican propiedades que la gramatica por si sola no puede garantizar: declaracion de identificadores, compatibilidad de tipos, manejo de constantes, validez de condiciones y dominios numericos en ciclos.

La fase semantica se ejecuta despues del parser. Por tanto, el analizador no decide si la cadena tiene forma valida; esa responsabilidad sigue siendo sintactica. Su tarea es recorrer el AST y comprobar si el programa construido tiene significado correcto dentro del lenguaje Claudio.

Convencion visual exigida

- **Rojo:** produccion sintactica original de la gramatica.
- **Fondo ambar:** regla semantica agregada con atributos, dominio y accion.
- Las reglas se implementaron como fase independiente sobre el AST: el parser construye el arbol y el analizador semantico lo recorre.

Cumplimiento del enunciado

Requisito del enunciado	Cumplimiento en Claudio
PDF con gramatica ampliada	Se presentan producciones originales en rojo y reglas semanticas agregadas en fondo ambar.
Minimo cinco reglas semanticas	Se especifican siete reglas: SEM-1 a SEM-7.
Atributos y dominios	Cada regla identifica atributos sintetizados/heredados y la restriccion que debe satisfacerse.
Acciones de validacion	Cada regla describe consulta a tabla, comparacion de tipos, reporte de error y continuacion del analisis.
Implementacion en compilador	Se documentan los archivos donde se implementan analizador, tabla, diagnosticos, endpoints y UI.
Bonus IA	Se describe OpenAI como capa adicional de explicacion sin reemplazar las reglas clasicas.

2. Resumen de reglas semanticas

ID	Regla	Nodo gramatical	Atributos principales	Dominio
SEM-1	Declaracion duplicada	decl_variable, def_funcion, parametros, clases	ID.nombre sintetizado; ambitoActual heredado	nombre no existe en el ambito actual
SEM-2	Identificador no declarado	sent_id, expresiones, imprimir, llamadas	ID.nombre sintetizado; ID.tipo sintetizado desde tabla	nombre existe en algun ambito activo
SEM-3	Constante reasignada	sent_id -> ID = expresion	ID.immutable heredado desde tabla	immutable == false para poder asignar
SEM-4	Tipo en declaracion	decl_variable	decl.tipo sintetizado; expr.tipo sintetizado	tipos iguales o entero -> real
SEM-5	Tipo en asignacion	sent_id -> ID = expresion	ID.tipo heredado; expr.tipo sintetizado	valor compatible con tipo declarado
SEM-6	Condicion booleana	sent_si, sent_mientras	cond.tipo sintetizado	cond.tipo == booleano
SEM-7	Limites numericos	sent_para, paso_opt	desde.tipo, hasta.tipo, paso.tipo sintetizados	cada tipo en {entero, real}

Todas las reglas reportan diagnosticos con fila, columna, lexema, regla SEM-N, mensaje descriptivo y sugerencia. El analisis continua tras cada error para acumular diagnosticos restantes y entregar una retroalimentacion completa al usuario.

3. Gramatica ampliada

3.1 Declaracion de variables

```
<decl_variable> -> (var | sea) <tipo> ID = <expresion>
```

SEM-1 - Declaracion duplicada en el mismo ambito

Atributos: ID.nombre es sintetizado desde el token ID; ambitoActual es heredado desde la pila de ambitos.

Dominio / restriccion: ID.nombre no debe existir en tablaSimbolos[ambitoActual].

Accion de validacion: Consultar la tabla del ambito actual. Si el nombre ya existe, reportar error con fila, columna, lexema y ubicacion de la primera declaracion. El analizador no se detiene.

SEM-4 - Tipo compatible en la inicializacion

Atributos: decl.tipo es sintetizado desde <tipo>; expr.tipo es sintetizado por inferencia sobre <expresion>.

Dominio / restriccion: decl.tipo == expr.tipo, o decl.tipo == real y expr.tipo == entero como conversion de ampliacion.

Accion de validacion: Inferir el tipo de la expresion, comparar contra el tipo declarado, reportar tipo esperado y tipo encontrado si no son compatibles, y registrar el simbolo para permitir recuperacion.

3.2 Sentencia iniciada por identificador

```
<sent_id> -> ID = <expresion> | ID ( <argumentos> ) | ID . ID <sufijo_id>
```

SEM-2 - Uso de identificador previamente declarado

Atributos: ID.nombre es sintetizado desde el token; ID.tipo se sintetiza desde la entrada encontrada en tabla.

Dominio / restriccion: ID.nombre debe existir en algun ambito activo.

Accion de validacion: Buscar desde el ambito mas interno hacia el global. Si no existe, reportar fila, columna, lexema y sugerir declaracion previa.

SEM-3 - Las constantes declaradas con sea no se reasignan

Atributos: ID.inmutable es heredado desde la entrada de tabla; ID.nombre es sintetizado.

Dominio / restriccion: Si la produccion es asignacion, ID.inmutable debe ser false.

Accion de validacion: Leer el atributo inmutable. Si fue declarado con sea, reportar error y continuar analizando la expresion asignada.

SEM-5 - Tipo compatible en asignacion posterior

Atributos: ID.tipo es heredado desde la tabla de simbolos; expr.tipo es sintetizado por inferencia.

Dominio / restriccion: El tipo de la expresion debe ser compatible con el tipo declarado del identificador.

Accion de validacion: Inferir expr.tipo, comparar con ID.tipo, reportar tipo esperado y tipo encontrado si falla.

3.3 Condicional y ciclo mientras

```
<sent_si> -> si <expresion> entonces <bloque> <rama_sino> fin_si  
<sent_mientras> -> mientras <expresion> hacer <bloque> fin_mientras
```

SEM-6 - La condicion debe ser booleana

Atributos: cond.tipo es sintetizado por inferencia sobre la expresion de condicion.

Dominio / restriccion: cond.tipo debe pertenecer al conjunto {booleano}.

Accion de validacion: Si la condicion se infiere como entero, real o cadena, reportar error en el token si o mientras y visitar el bloque para conservar recuperacion.

3.4 Ciclo para

```
<sent_para> -> para ID desde <expresion> hasta <expresion> <paso_opt> hacer <bloque>  
fin_para  
<paso_opt> -> paso <expresion> | epsilon
```

SEM-7 - Limites y paso numericos

Atributos: desde.tipo, hasta.tipo y paso.tipo son atributos sintetizados desde sus expresiones.

Dominio / restriccion: Cada tipo debe pertenecer a {entero, real}. Si paso_opt es epsilon no se valida paso.

Accion de validacion: Declarar la variable de ciclo como entero en un ambito local, validar desde/hasta/paso y reportar cada violacion sin detener el recorrido.

3.5 Funciones, parametros y ambitos

```
<def_funcion> -> funcion <tipo_retorno> ID ( <parametros> ) hacer <bloque> fin_funcion  
<parametro> -> <tipo> ID
```

SEM-1 aplicada a funciones y parametros

Atributos: ID.nombre es sintetizado; ambitoActual es heredado. En parametros tambien se sintetiza parametro.tipo.

Dominio / restriccion: Una funcion no debe duplicarse en el ambito donde se declara; un parametro no debe duplicarse dentro de la misma firma/ambito.

Accion de validacion: Registrar la funcion como simbolo de tipo funcion; entrar a un nuevo ambito para parametros y cuerpo; reportar duplicados con SEM-1.

4. Tabla de simbolos

La tabla de simbolos almacena los atributos requeridos por las reglas semanticas y se implementa con una pila de ambitos. Cada ambito es un diccionario de nombres a entradas de simbolo.

Atributo	Uso semantico
nombre	Deteccion de duplicados y busqueda de identificadores.
tipo	Comparacion de tipos en declaracion, asignacion, condicion y ciclos.
inmutable	Validacion de constantes declaradas con sea.
inicializado	Estado disponible para reglas futuras y trazabilidad.
ambito	Resolucion de nombres desde el ambito interno al global.
fila, columna	Reporte exacto del diagnostico al usuario.

5. Implementacion en el compilador

La implementacion separa fases: el parser descendente recursivo construye el AST y el analizador semantico realiza una pasada posterior. Esta arquitectura cumple el requisito de separacion entre analisis sintactico y semantico.

Archivo	Responsabilidad
backend/semantico.py	AnalizadorSemantico, TablaSimbolos, SemanticDiagnostic y reglas SEM-1 a SEM-7.
backend/main.py	Endpoint POST /api/semantico y endpoint POST /api/sugerencias-ia-semantico.
backend/ai_suggestions.py	Integracion OpenAI para explicar diagnosticos semanticos ya detectados.
frontend/src/app/page.tsx	Modo semantico, ejecucion del endpoint y conexion con sugerencias IA.
frontend/src/components/errors-view.tsx	Vista de errores semanticos con regla, fila, columna, lexema y sugerencia.
frontend/src/components/symbol-table-view.tsx	Tabla de simbolos visible: nombre, tipo, inmutable, ambito, fila y columna.
backend/test_quiz4_semantico.py	Pruebas unitarias del caso integral, caso valido, endpoint y fallback OpenAI.

Flujo de ejecucion

- El usuario envia codigo a POST /api/semantico.
- El backend ejecuta analisis lexico y sintactico. Si existe AST, invoca AnalizadorSemantico.
- El analizador recorre nodos de declaracion, asignacion, condiciones, ciclos, funciones y expresiones.
- Cada error se agrega como SemanticDiagnostic; no se lanza una excepcion que detenga el analisis.
- La respuesta incluye valido, errores_semanticos, total_errores_semanticos y tabla_simbolos.

6. Bonus opcional: integracion OpenAI

Se implementa la modalidad A del enunciado: IA como capa adicional de explicacion. Las reglas classicas SEM-1 a SEM-7 siguen siendo la fuente de verdad; OpenAI no reemplaza el analisis por atributos.

- Entrada al modelo: codigo fuente y diagnosticos semanticos ya detectados.
- Salida esperada: explicacion para el usuario, causa probable, correccion sugerida, mini ejemplo y nivel de confianza.
- Endpoint: POST /api/sugerencias-ia-semantico.
- Configuracion: OPENAI_API_KEY, OPENAI_MODEL, OPENAI_TIMEOUT_SECONDS y OPENAI_MAX_ERRORS.
- Fallback: si no hay clave, el compilador sigue funcionando y retorna estado no_disponible junto con la sugerencia deterministica.

Ejemplo de error enriquecido por IA

```
var entero x = "hola"
```

Diagnostico clasico: SEM-4, tipo incompatible en declaracion: x fue declarado como entero pero el valor asignado es cadena. La capa OpenAI puede explicar que debe cambiarse el literal a un entero o ajustar el tipo declarado a cadena.

7. Casos de validacion

Caso integral con recuperacion

```

var entero x = "hola"
var entero x = 2
x = "texto"
sea entero c = 1
c = 2
imprimir(fantasma)
si x entonces
    imprimir(x)
fin_si
para i desde "a" hasta 10 paso falso hacer
    imprimir(i)
fin_para

```

- SEM-1: x duplicado en el mismo ambito.
- SEM-2: fantasma no declarado.
- SEM-3: c fue declarado con sea y se reasigna.
- SEM-4: x declarado entero con valor cadena.
- SEM-5: x recibe cadena en asignacion posterior.
- SEM-6: condicion si x no es booleana.
- SEM-7: desde/paso no numericos en para.

Caso valido

```

funcion entero suma(entero a, entero b) hacer
    var entero resultado = a + b
    retornar resultado
fin_funcion

var entero total = 10
total = 20
si total > 0 entonces
    imprimir(total)
fin_si

```

Resultado esperado: sin errores semanticos y tabla con suma, a, b, resultado y total.

8. Evidencia de validacion

La validacion se planteo sobre tres frentes: comportamiento del analizador, respuesta de la interfaz y consistencia entre la gramatica documentada y las reglas implementadas.

- Caso integral: un mismo programa activa SEM-1 a SEM-7 y demuestra recuperacion semantica.
- Caso valido: un programa con funcion, parametros, asignacion compatible y condicion booleana no produce errores semanticos.
- Tabla de simbolos: la UI muestra nombre, tipo, inmutabilidad, ambito, fila y columna de cada entrada registrada.
- API semantica: el backend retorna bandera de validez, lista de diagnosticos y tabla de simbolos estructurada.
- IA: cuando OpenAI esta configurado, los diagnosticos clasicos se enriquecen con una explicacion pedagogica y una correccion sugerida.

La prueba puntual de **var entero x = "hola"** debe producir SEM-4, porque el tipo declarado es entero y el literal asignado es cadena. Este caso evidencia que el analizador compara tipos, no solo tokens o estructura sintactica.

9. Conclusiones

La ampliación semántica convierte a Claudio en un compilador más completo: ya no se limita a aceptar programas bien formados, sino que también valida que los nombres, tipos, constantes, condiciones y ciclos tengan coherencia dentro del lenguaje. Esta diferencia es central en teoría de compiladores, porque separa claramente la forma del programa de su significado.

La tabla de símbolos es el componente que conecta la gramática con el comportamiento real del compilador. Al almacenar tipo, ámbito, inmutabilidad y ubicación fuente, permite detectar errores que no son visibles desde la sintaxis, como usar una variable no declarada, duplicar nombres en el mismo ámbito o reasignar una constante.

El modo de recuperación mejora la experiencia de uso del compilador: en lugar de detenerse ante el primer fallo, Claudio acumula diagnósticos y entrega una visión más completa de los problemas del programa. Esto hace que el analizador sea más útil para depuración, demostración en clase y validación automatizada.

La integración con OpenAI se mantiene como complemento, no como reemplazo de las reglas formales. Las reglas SEM-1 a SEM-7 siguen siendo deterministas y verificables; la IA agrega una capa de explicación en lenguaje natural que ayuda al usuario a entender la causa del error y una posible corrección.

- Se documentan siete reglas semánticas, superando el mínimo solicitado.
- Cada regla incluye atributos, dominio y acción de validación.
- La implementación respeta la separación de fases: parser primero, analizador semántico después.
- Los diagnósticos incluyen regla, fila, columna, lexema, mensaje y sugerencia.
- El bonus de OpenAI queda integrado como asistencia explicativa sobre errores ya detectados.